BAOUCH Naguib Mohammed
SELMANI Mohammed

Compte Rendu Tp1

1. Objectif du TP

Ce travail pratique a pour objectif principal de nous permettre de découvrir et de prendre en main l'environnement Vivado, afin de concevoir, simuler et valider des circuits logiques simples sur FPGA. Il permet d'aborder les étapes essentielles d'un projet FPGA, depuis la création du projet jusqu'à la vérification fonctionnelle, en passant par la description VHDL et l'utilisation de différentes méthodes de simulation.

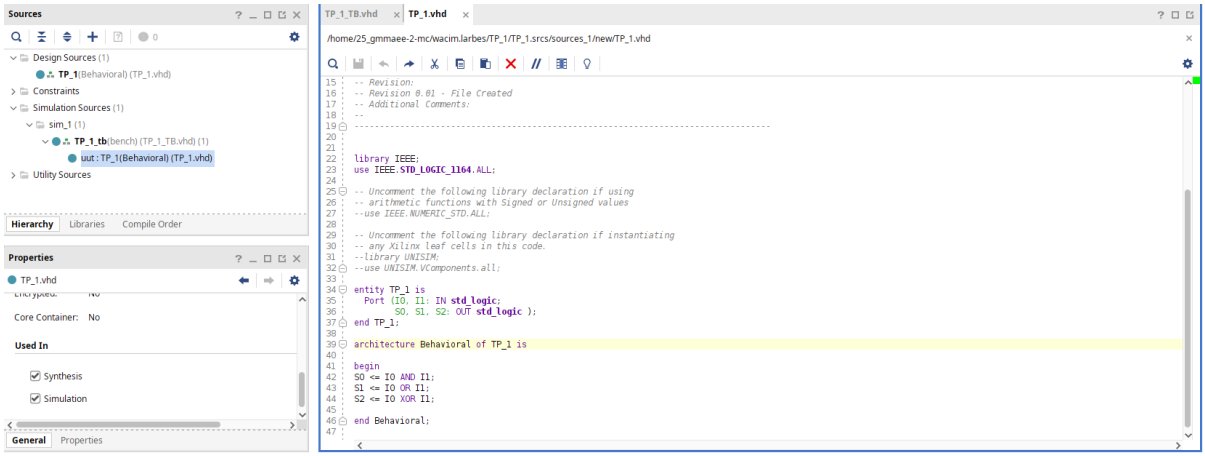
2. Création du projet sous ²Vivado

Dans un premier temps, le logiciel Vivado a été lancé et un nouveau projet a été créé à l'aide de l'assistant *Create Project*. Le type de projet sélectionné est **RTL Project**, avec le langage de description matérielle **VHDL**. Aucune contrainte n'a été ajoutée lors de cette étape.

Le composant cible retenu est le **xc7z020clg484-1**, correspondant à la carte FPGA utilisée durant le TP. Une fois le projet configuré, l'interface de Vivado offre l'accès aux différentes fenêtres telles que le navigateur de projet, la gestion des sources, les propriétés, la zone de visualisation ainsi que la console.

3. Création et simulation d'une porte logique

3.1 Description VHDL

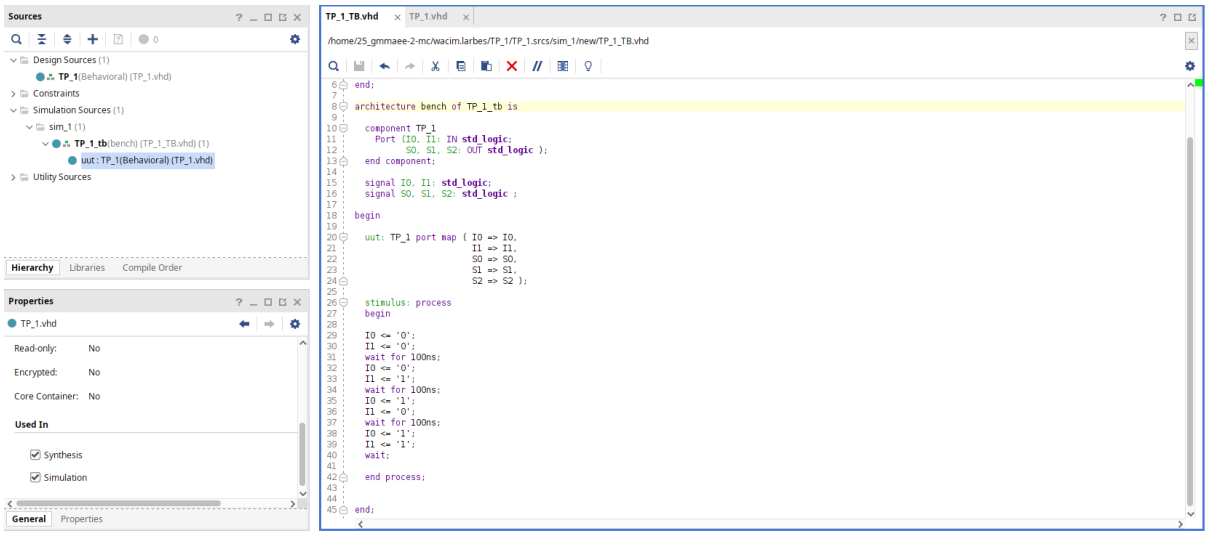


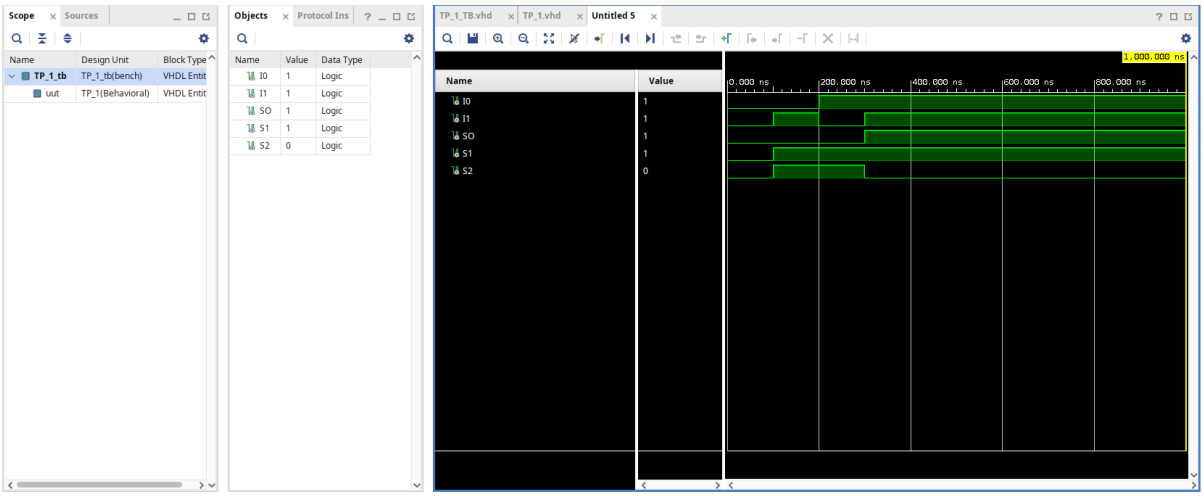
Une description VHDL a été réalisée afin d'implémenter des portes logiques combinatoires à deux entrées. Les portes **AND**, **OR** et **XOR** ont été choisies comme exemples.

Le code VHDL se compose :

- d'une entité définissant les entrées *I0*, *I1* et les sorties *S0* (AND), *S1* (OR) et *S2* (XOR) ;
- d'une architecture de type **RTL** implémentant les équations logiques associées.

3.2 Simulation





Une simulation comportementale a ensuite été réalisée afin d'observer le fonctionnement du circuit. Initialement, les signaux apparaissent dans un état non initialisé (*U*).

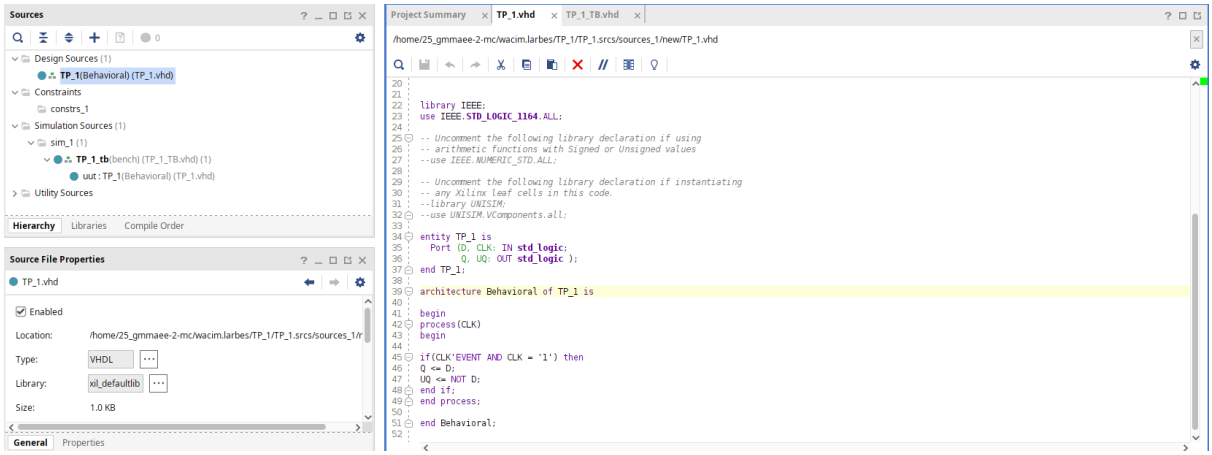
Trois méthodes de simulation ont été mises en œuvre :

- le forçage manuel des entrées afin de tester différentes combinaisons ;
- le forçage d'une horloge pour observer automatiquement plusieurs transitions ;
- l'utilisation d'un testbench VHDL, méthode la plus complète et la plus fiable.

Les résultats obtenus sont conformes aux tables de vérité théoriques des portes logiques.

4. Réalisation d'une bascule D

4.1 Table de vérité



La bascule D fonctionne de manière synchrone sur le front montant de l'horloge. La sortie *Q* prend la valeur de l'entrée *D* uniquement lors de ce front.

4.2 Vérification par forçage

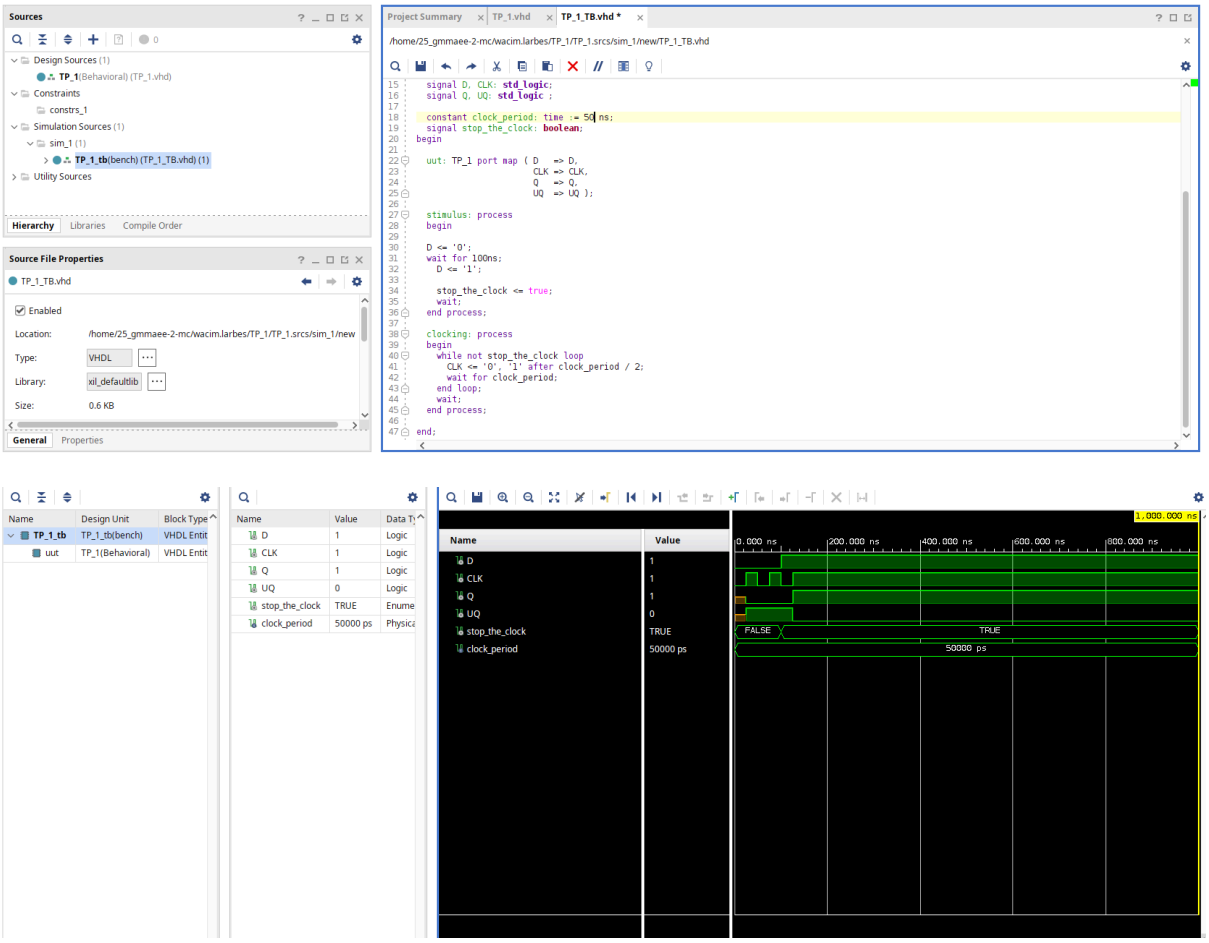
Le comportement de la bascule a été validé par :

- le forçage de constantes sur l'entrée *D* ;
- le forçage d'une horloge afin d'observer les changements d'état.

Les résultats observés correspondent au fonctionnement attendu d'une bascule D.

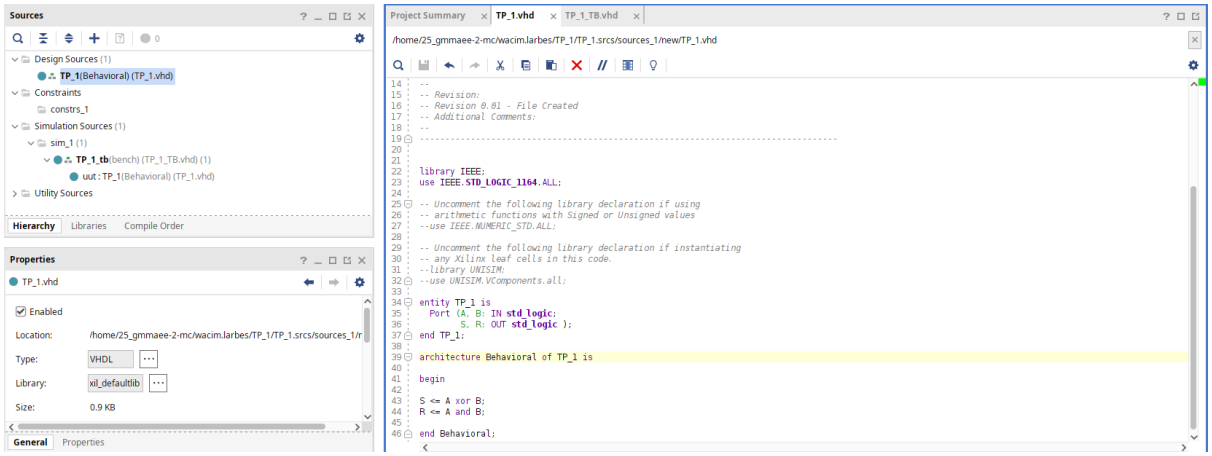
4.3 Simulation avec testbench

Un testbench VHDL a été conçu pour appliquer différentes valeurs de *D* synchronisées avec l'horloge. La simulation confirme que la sortie *Q* ne change qu'au front montant de l'horloge.



5. Additionneur complet 1 bit

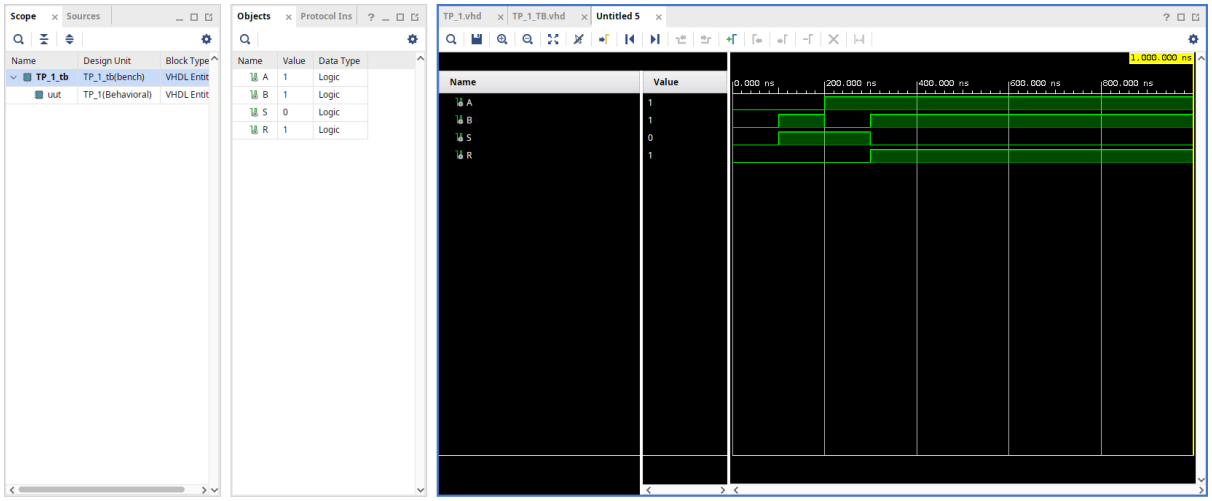
5.1 Principe



L'additionneur complet 1 bit est constitué de :

- deux entrées *A* et *B* ;
- une sortie somme *S* ;
- une sortie retenue *R*.

5.2 Simulation



Le module VHDL de l'additionneur complet a été implémenté, puis testé à l'aide d'un testbench couvrant toutes les combinaisons possibles des entrées. Les résultats montrent que :

- la somme *S* correspond correctement à l'addition binaire ;
- la retenue *R* est générée lorsque nécessaire.

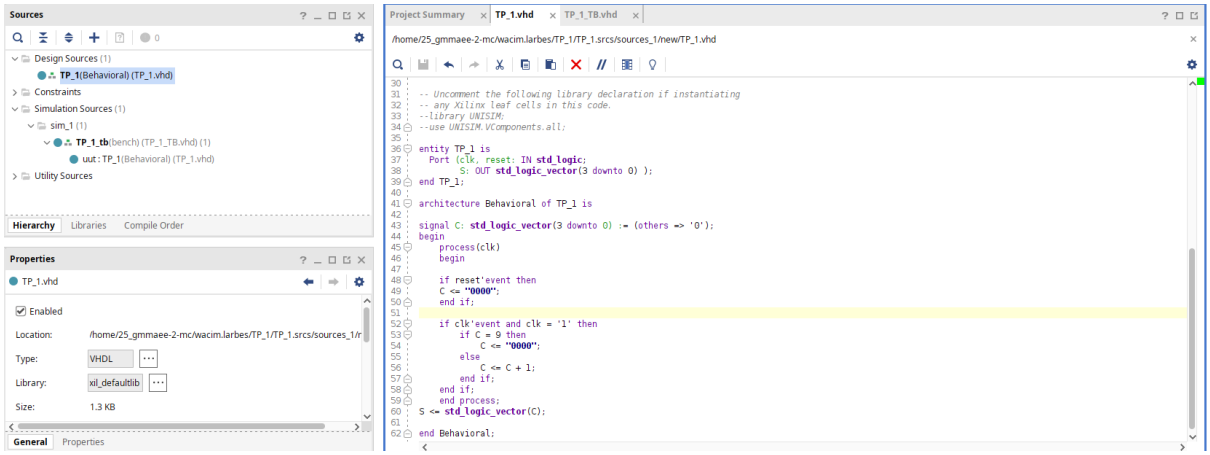
Le comportement obtenu est conforme à la table de vérité théorique.

5.3 Schéma équivalent

L'additionneur complet peut être représenté par une association de portes logiques **XOR**, **AND** et **OR**.

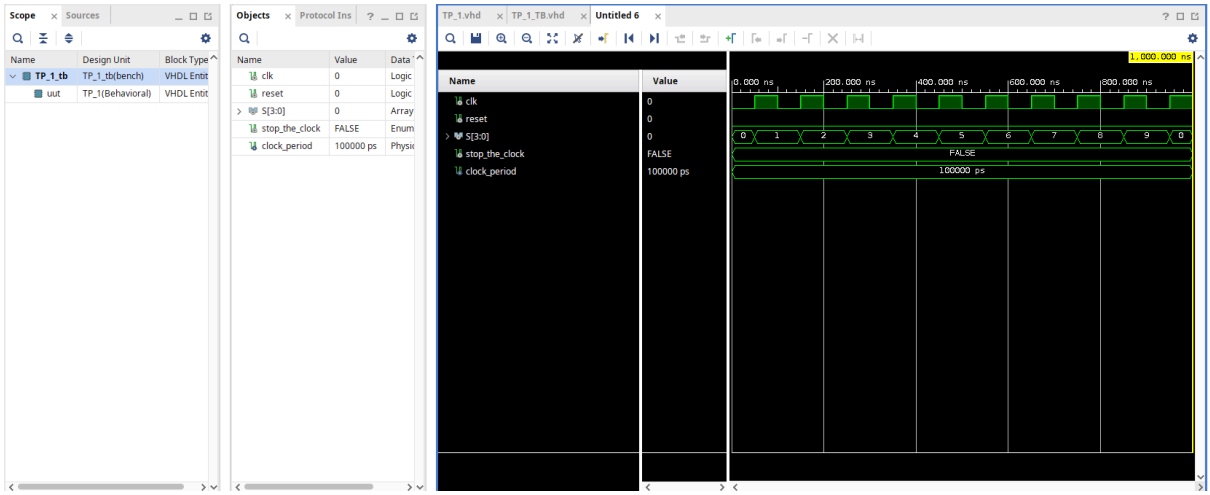
6. Compteur synchrone modulo 10 avec reset

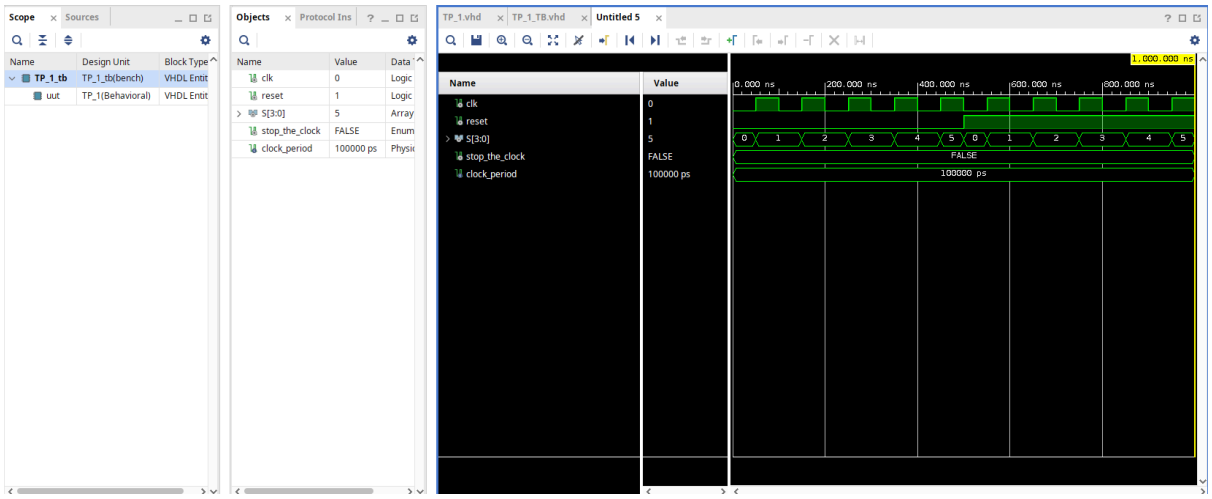
6.1 Description



Un compteur synchrone modulo 10 a été conçu. Il s'incrémente à chaque front montant de l'horloge et se remet à zéro lorsque la valeur 9 est atteinte ou lorsque le signal de *reset* est activé.

6.2 Simulation





Le testbench génère une horloge périodique ainsi que des impulsions sur le signal de reset. La simulation met en évidence que :

- le compteur évolue correctement de 0 à 9 ;
- le reset force le retour à zéro.

6.3 Schéma équivalent

Le compteur peut être modélisé par un ensemble de bascules D associées à une logique combinatoire assurant la remise à zéro.

7. Conclusion générale

Ce travail pratique nous a permis de nous familiariser avec l'outil Vivado et de mieux comprendre les principales étapes de conception de circuits logiques sur FPGA. La simulation, et plus particulièrement l'utilisation de testbenchs, s'est révélée essentielle pour valider le bon fonctionnement des différents modules. Les circuits réalisés (portes logiques, bascule D, additionneur et compteur) constituent une base solide pour la conception de systèmes FPGA plus complexes.